CASE STUDY

REAL-TIME FOOD ORDER TRACKING USING APACHE KAFKA

A Zomato-Inspired Event Streaming System

Name: DHARUN P

USN: 23BTRCL014

Assigned Tool: Apache Kafka

Batch: Batch 1

Submitted as a case study on real-time data streaming

1. Introduction

Food-delivery platforms process many events every second: customers place orders, restaurants accept and prepare them, delivery partners collect them, and customers receive them. These events need to reach tracking, notification, and analytics systems quickly.

Apache Kafka is a distributed event-streaming platform that stores events in topics and lets multiple applications read them. This case study describes a small educational simulation of a food-delivery order pipeline. It is inspired by a platform such as Zomato; it is not a description of Zomato's private production infrastructure.

2. Problem Statement

A food-delivery application needs to distribute order-status updates to multiple services with low delay. A system that passes each update directly from one application to another can become difficult to maintain as more services are added.

The proposed solution is to publish each order event to Kafka. Tracking, notifications, and analytics can then consume the stream independently. This reduces tight coupling between the application that produces events and the applications that use them.

3. Aim and Objectives

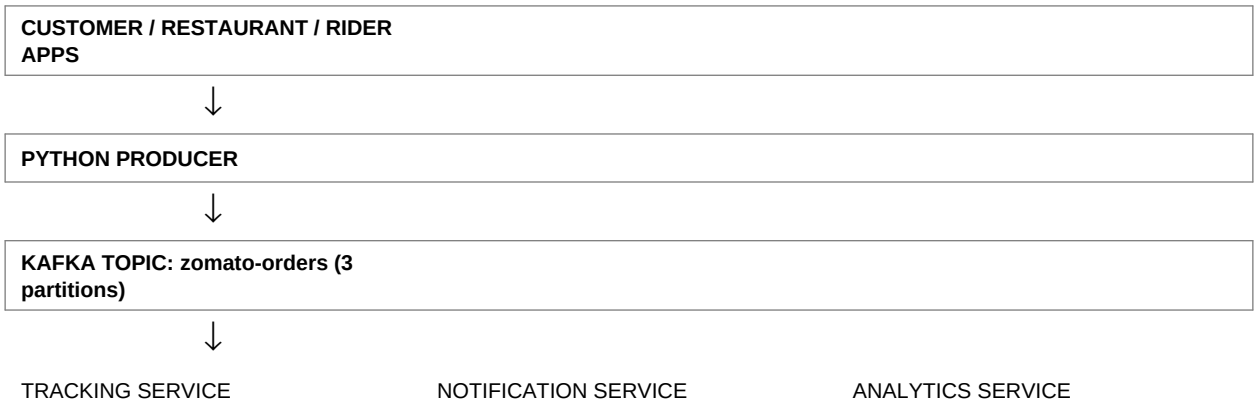
Aim: To demonstrate how Apache Kafka can stream food-order status events in near real time.

Objectives: (1) create a Kafka topic for order events; (2) publish sample order updates using a producer; (3) read events using consumers; (4) understand partitions, keys, and offsets; and (5) compare load sharing within one consumer group with independent consumption by different groups.

4. Tools and Requirements

• Apache Kafka (ZooKeeper-based setup for this lab example)\n• Java JDK 17 or later\n• Python 3.11 or later\n• Python Kafka client library compatible with the installed broker\n• Windows terminal / Command Prompt\n• A producer script and a consumer script containing sample order events

5. System Architecture



Customer / Restaurant / Delivery App → Python Producer → Kafka Topic: zomato-orders → Consumer Groups

Consumer group examples: tracking-service (live status) • notification-service (customer alerts) • analytics-service (reports and dashboards).

Kafka stores the records in partitions. The order_id is used as the message key so events for one order can be routed consistently to the same partition. Each record receives an offset within its partition.

6. Kafka Concepts in the Case Study

Producer: The application that sends an order event to Kafka.

Topic: A named stream, for example zomato-orders, that stores order events.

Partition: A segment of a topic that supports parallel processing. This demonstration can use three partitions.

Message key: order_id, such as ORD4501. Using the same key helps keep an order's events in the same partition.

Offset: A record's position within a partition. Offsets increase as records are appended.

Broker: The Kafka server that stores and serves records.

Consumer: An application that reads records from a topic.

Consumer group: A set of consumers working together. Consumers in one group share partition work; separate groups can each read the stream independently.

ZooKeeper: In the ZooKeeper-based Kafka configuration used by this lab, it coordinates cluster metadata and broker state.

7. Implementation Procedure

Step 1 — Prepare the environment. Install a supported JDK and Kafka distribution, then install the Kafka Python client into the same Python interpreter that will run the scripts.

Step 2 — Start ZooKeeper, if using the ZooKeeper-based distribution, and leave its terminal open.

Step 3 — Start the Kafka broker in another terminal and wait until the startup message confirms it is running.

Step 4 — Create a topic named zomato-orders with three partitions and replication factor one for a single-broker local demonstration. Verify the topic configuration with the topic describe command.

Step 5 — Start the consumer with group ID tracking-service. It should wait for records from the topic.

Step 6 — Run the producer. It publishes a sequence of order events such as PLACED, CONFIRMED, PREPARING, PICKED_UP, and DELIVERED.

Step 7 — Observe the producer and consumer output. Record the key, partition, offset, and order status for several messages.

Step 8 — Start a second consumer using the same group ID to observe partition sharing. Then use a different group ID, such as analytics-service, to demonstrate independent consumption.

8. Example Event Stream

An order can generate multiple events over its lifecycle. The following is illustrative sample data, not a live feed from Zomato.

order_id	status	restaurant	meaning
ORD4501	PLACED	Paradise Biryani	Customer submits order
ORD4501	CONFIRMED	Paradise Biryani	Restaurant accepts
ORD4501	PREPARING	Paradise Biryani	Food is being prepared
ORD4501	PICKED_UP	Paradise Biryani	Delivery partner collects
ORD4501	DELIVERED	Paradise Biryani	Order reaches customer

9. Sample Output and Interpretation

Example records: (key=ORD4501, partition=2, offset=0, status=PLACED); (key=ORD4501, partition=2, offset=3, status=CONFIRMED); (key=ORD4501, partition=2, offset=7, status=PREPARING); (key=ORD4501, partition=2, offset=9, status=PICKED_UP).

The specific partition and offsets depend on the topic configuration, key partitioner, and messages already present. The important observation is that records with the same key are consistently routed to the same partition when key-based partitioning is configured.

10. Consumer Group Demonstration

Same group: When two consumers join tracking-service, Kafka assigns the topic partitions between active consumers in that group. Each partition is assigned to one consumer within the group at a time, so the group shares the workload. If there are more consumers than partitions, some consumers may be idle.

Different groups: A consumer in analytics-service has its own group position and can read the same events independently of tracking-service. This allows tracking, notifications, and analytics to use the same event stream for different purposes.

11. Observations

• The producer publishes order-status events to a named Kafka topic.

• The consumer receives events without requiring a direct connection to the producer application.

• The key can keep one order's events on the same partition, preserving their partition order.

• Offsets identify records within each partition and help consumers track their reading position.

• Consumers in the same group share partitions; different groups can independently consume the stream.

• Kafka can act as a buffer between event-producing applications and downstream services.

12. Challenges and Troubleshooting

Broker does not start: check the broker configuration, log directory permissions, and whether another process is using the configured ports.

Python reports NoBrokersAvailable: confirm that Kafka is running, the bootstrap server and port are correct, and the Python client is compatible with the broker.

Producer times out: verify the listener and advertised listener addresses so the client can reach the broker using the advertised address.

Python cannot import the Kafka library: install it using the same interpreter that runs the script, for example python -m pip install <compatible-client-package>.

Records appear in an unexpected partition: check the message key, partition count, and partitioner. Partition IDs and offsets are examples and may differ between runs.

13. Benefits and Limitations

Benefits: Kafka decouples producers from consumers, supports multiple independent consumer groups, stores ordered records within partitions, and can scale through partitioning and additional brokers.

Limitations of this lab: a single local broker with replication factor one is not fault tolerant. The sample events are simulated, and the demonstration does not implement real payment, customer data, restaurant systems, delivery GPS, or production security. A real deployment needs monitoring, access controls, replication, retention policies, error handling, and careful delivery semantics.

14. Conclusion

This case study demonstrates how Apache Kafka can support a real-time food-order tracking workflow. Order updates are produced as events, stored in a Kafka topic, and consumed by separate applications. Keys, partitions, and offsets explain how Kafka organizes records, while consumer groups show how services can share processing or independently read the same stream. The example provides a foundation for understanding event-driven systems used in food delivery, logistics, and other real-time applications.

15. References

• Apache Kafka documentation: https://kafka.apache.org/documentation/

• Apache Kafka quickstart: https://kafka.apache.org/quickstart

• Python Kafka client documentation for the specific client package used in the lab.

Note: This report is an educational simulation and does not claim access to Zomato's internal systems.